Attendance

Computer Organisation

Topic 4
Digital Logic (Part 2)

Stallings, Computer Organization & Architecture (9th Ed.), Chap. 11

Outline

- Sequential Circuits
  - Flip-Flops
  - Registers
  - Counters
- Programmable Logic Devices
  - Programmable Logic Array
  - Field-Programmable Gate Array

Introduction

- Combinational circuits
  - Implement **functions** of a computer
  - Outputs depend entirely on the present input
  - **No memory** (except for ROM)

Introduction

- **Storage** elements
  - Devices that are capable of storing binary information (which defines the state of the sequential circuit)

Sequential Circuit

- Current output depends on the **current input** and the **current state** of the circuit
- Consists of a combinational circuit to which storage elements are connected to form a **feedback path**

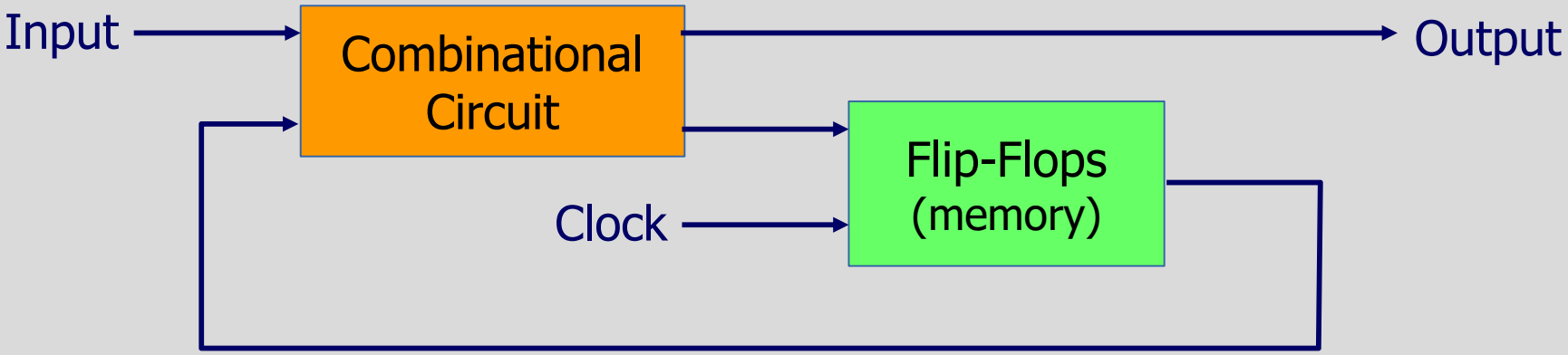
Flip-Flops

- **Simplest** sequential circuit
- It has the capability to **"store" information** (1-bit memory)

Flip-Flops

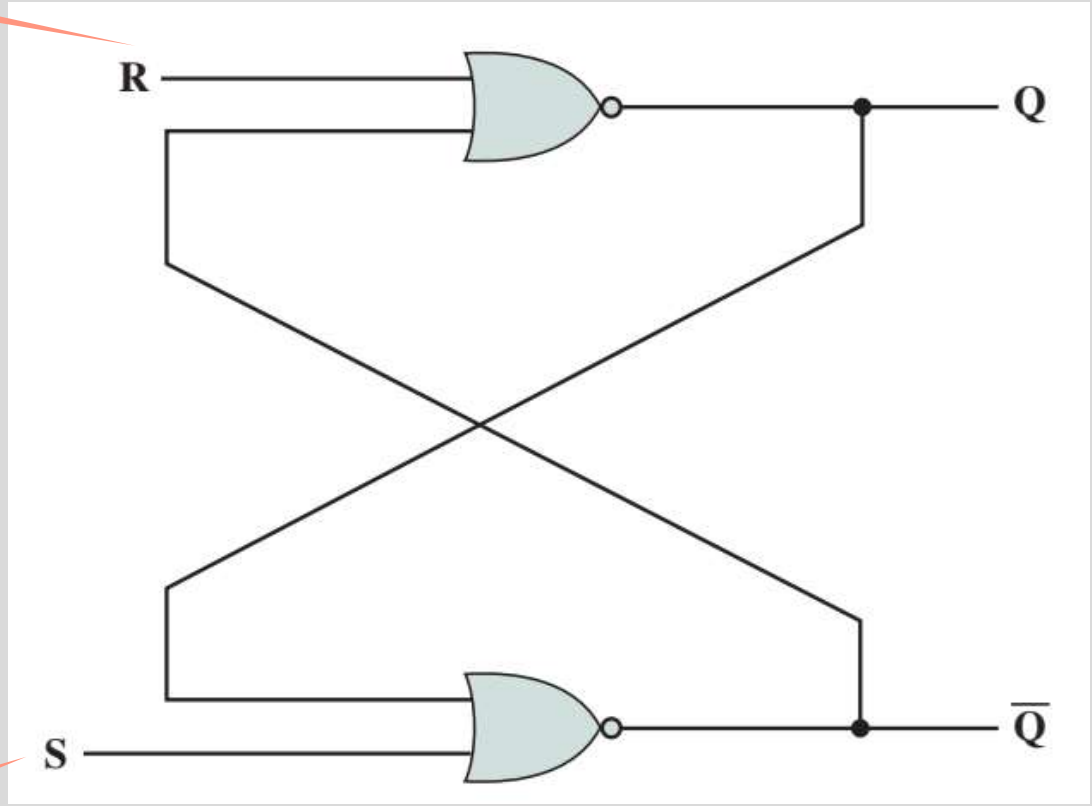
- A **bistable** device
  - Exists in one of two states
  - In the absence of the input remains in that state
- With **two outputs**, Q & $\overline{Q}$, which are the complement of each other

Flip-Flops


Input
Combinational Circuit
Output
Flip-Flops (memory)
Clock

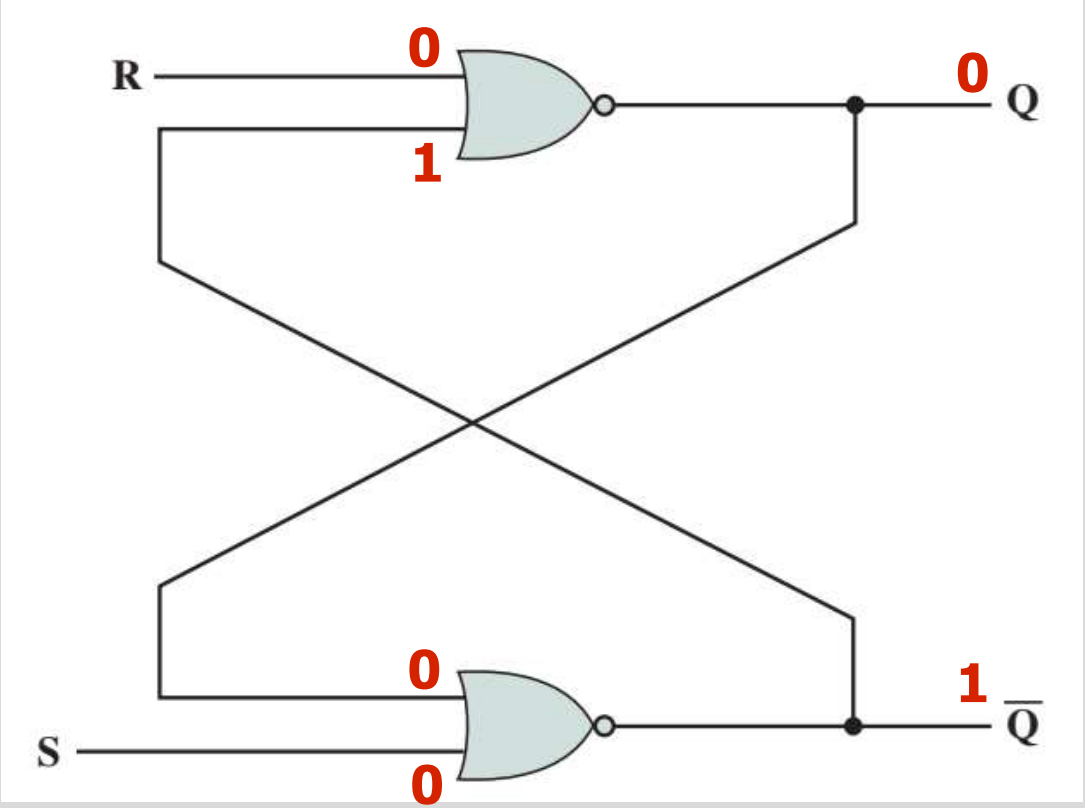
S-R Latch

- 2 inputs
  - S (Set) and R (Reset)
- 2 outputs
  - Q and $\overline{Q}$
- 2 NOR gates connected in a feedback manner



S-R Latch

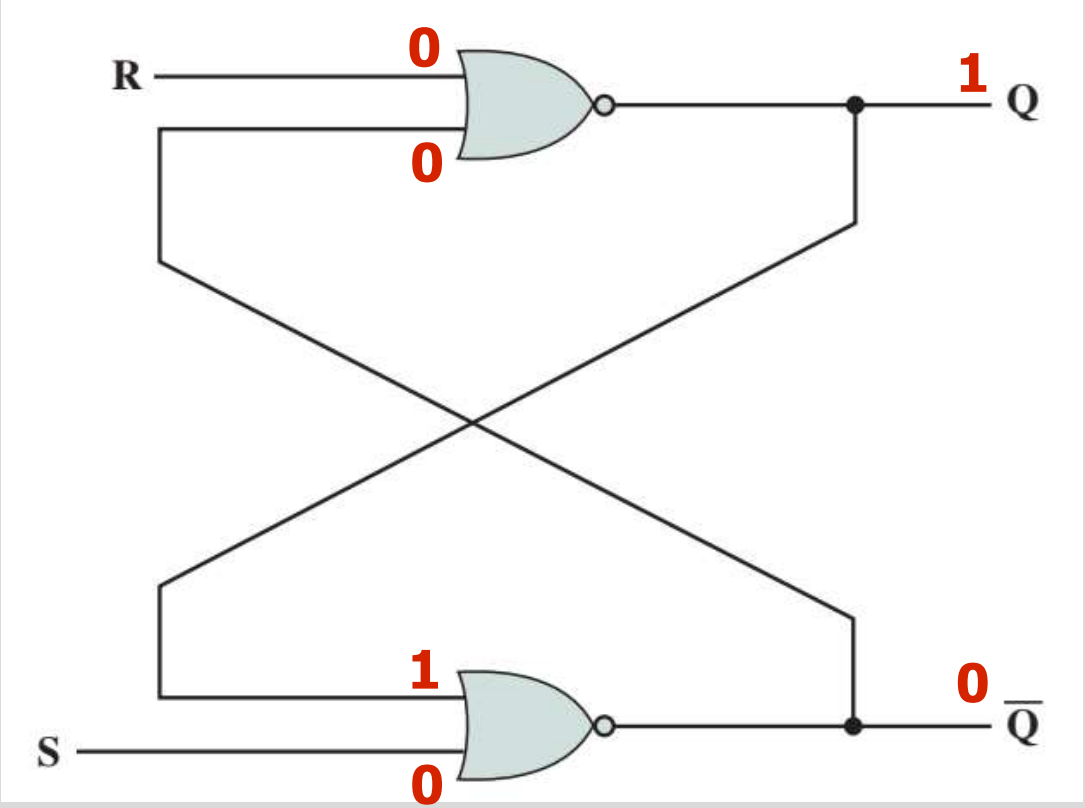
- S =0, R=0, Q=0
  - Inputs to lower NOR gate are Q=0, S=0



Flip-flops remains stable in this state

S-R Latch

- S =0, R=0, Q=1
  - Inputs to lower NOR gate are Q=1, S=0



Flip-flops remains stable in this state

Setting the S-R Latch

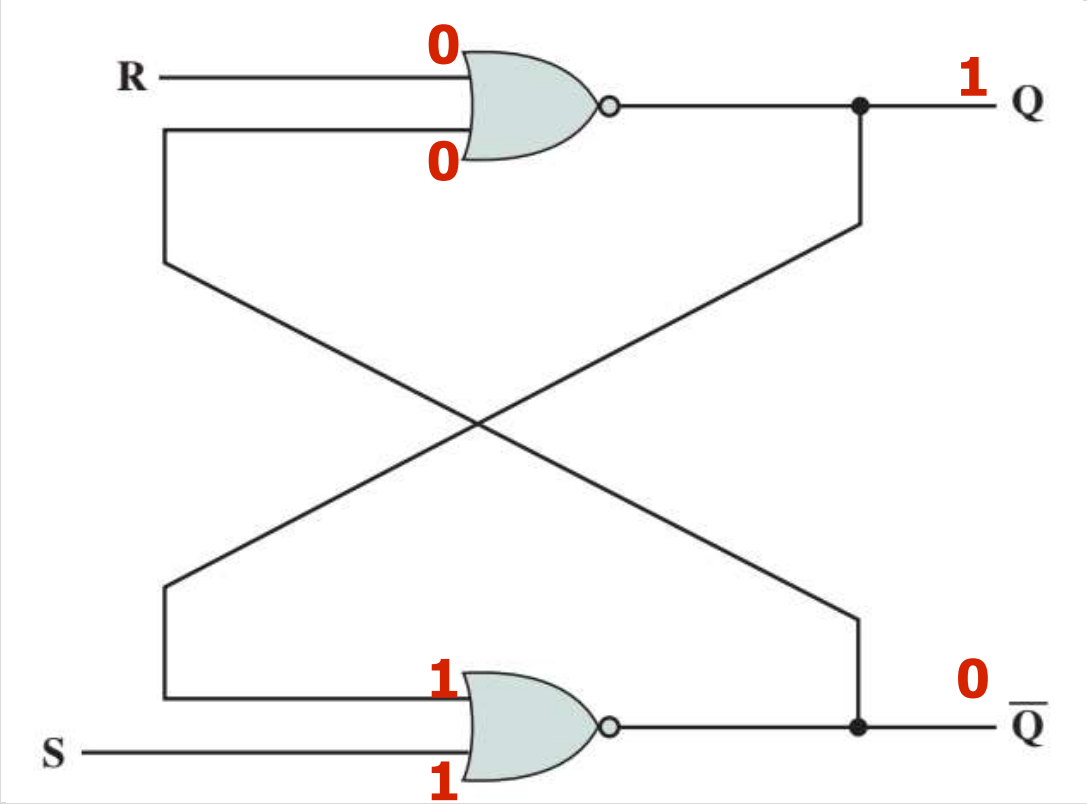
S-R Latch

Let's say initially

- S=0, R=0, Q=0, $\overline{Q}$=1

Then S=1

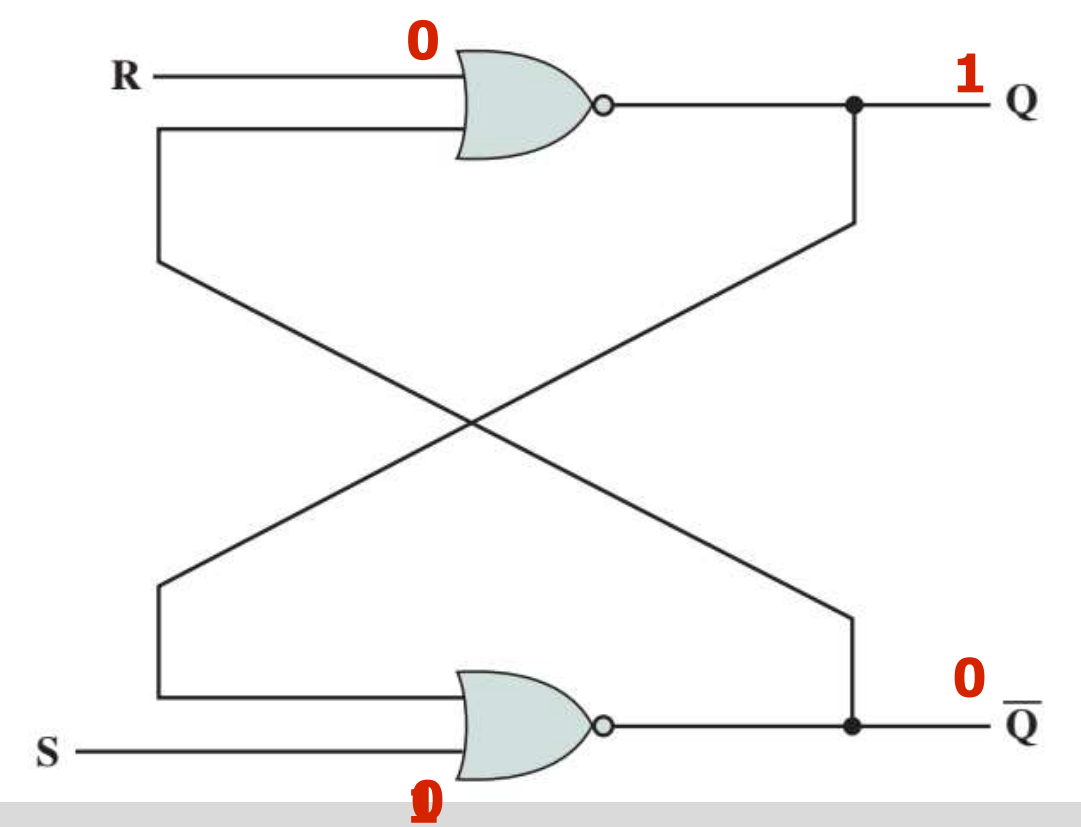
- ✓ Inputs to lower NOR gate are S=1, Q=0
- ✓ After Δt, output of lower NOR gate will be Q=0
- ✓ Inputs to the upper NOR gate become R=0, Q=0
- ✓ After another Δt, output Q=1



S-R Latch

After that, if S=0

- Inputs to lower NOR gate are S=0, Q=1
- After Δt, output of lower NOR gate will be Q=0
- Inputs to the upper NOR gate become R=0, Q=0
- After another Δt, output Q=1



Reset the S-R Latch

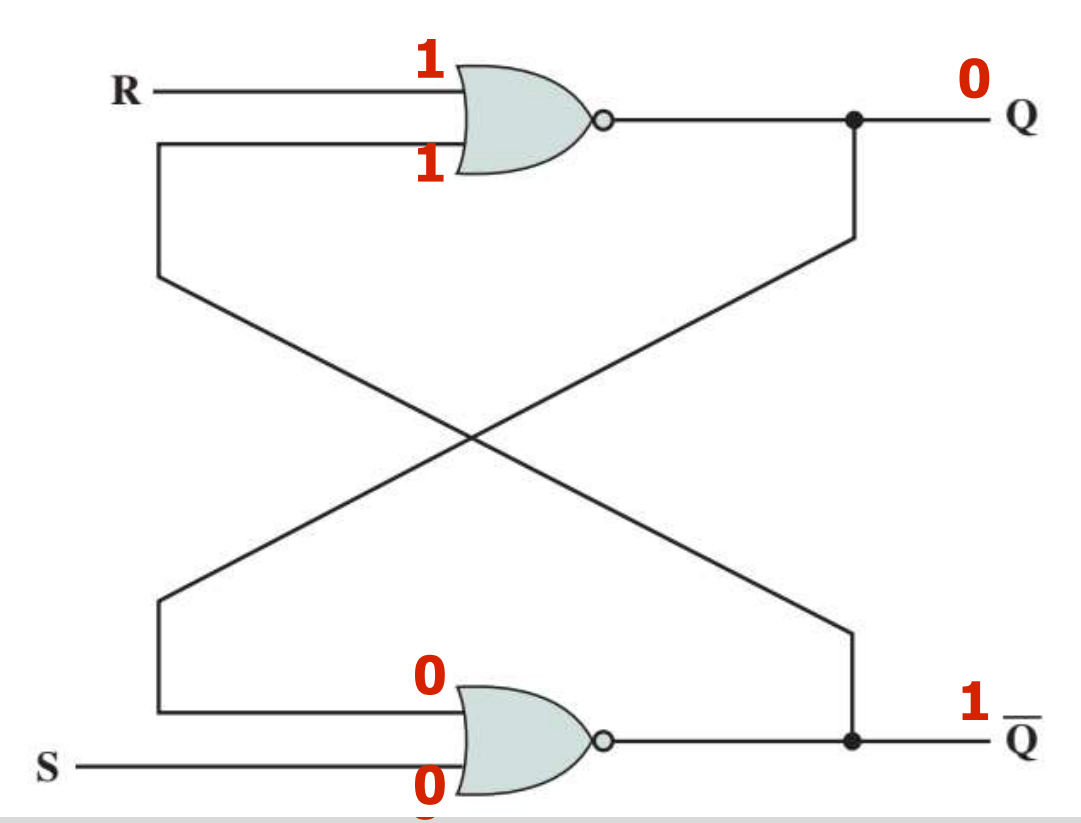
S-R Latch

Initially S=0, R=0, Q=1, Q=0

R=1

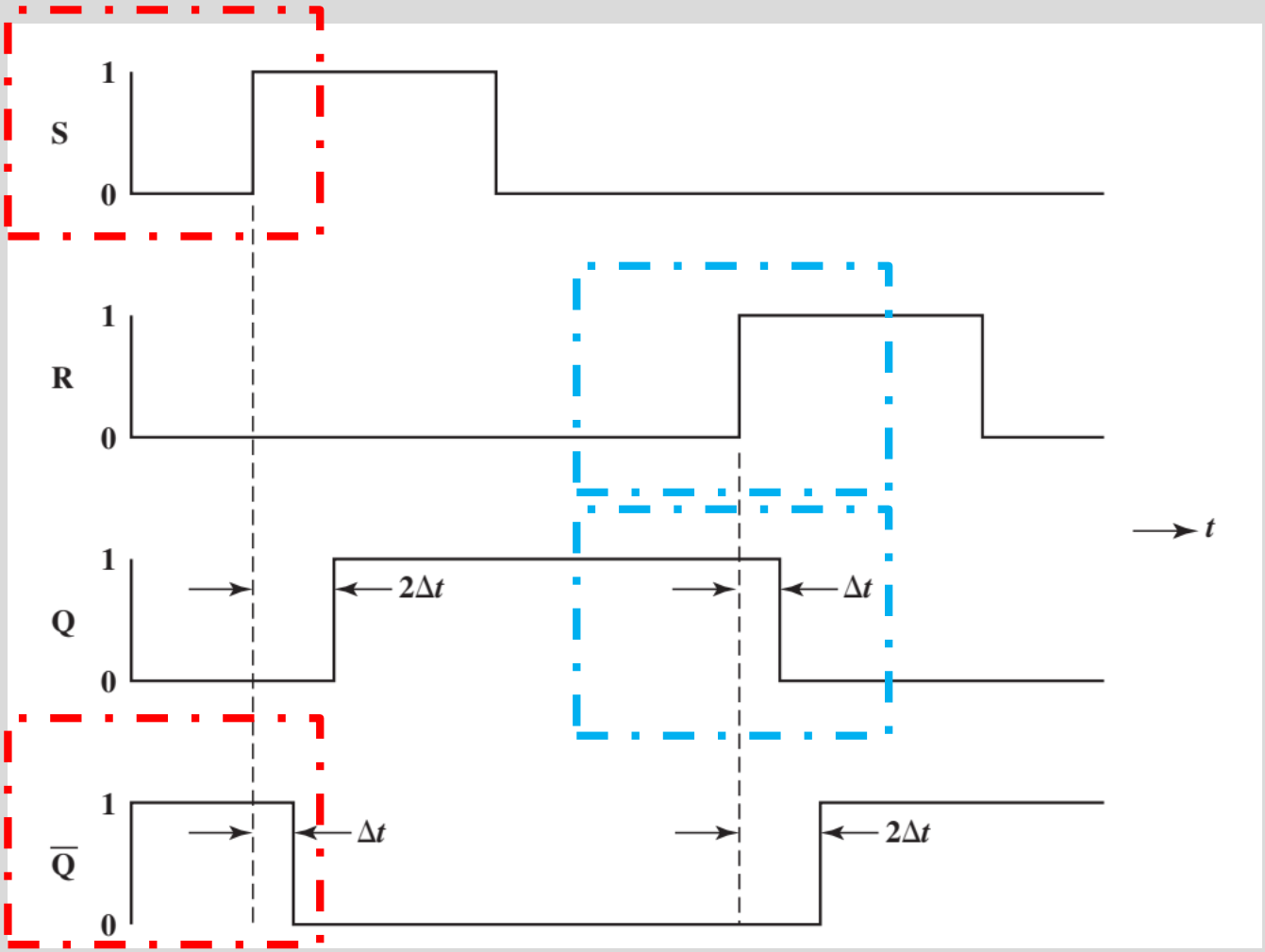
- Inputs to upper NOR gate are R=1, Q=0
- After Δt, output of upper NOR gate will be Q=0
- Inputs to the lower NOR gate become S=0, Q=0
- After another Δt, output Q=1

- After that, if R=0
  - Q=0, Q=1



S-R Latch

NOR S–R Latch Timing Diagram



S-R Latch

- Summary
  - S writes a 1 to Q
  - R writes a 0 to Q
  - S=1 and R=1 at the same time are not allowed
    - Inconsistent output

(a) Characteristic Table

Current Inputs	Current State	Next State
SR	Q_n	Q_{n+1}
00	0	0
00	1	1
01	0	0
01	1	0
10	0	1
10	1	1
11	0	–
11	1	–

(b) Simplified Characteristic Table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	–

S-R Latch

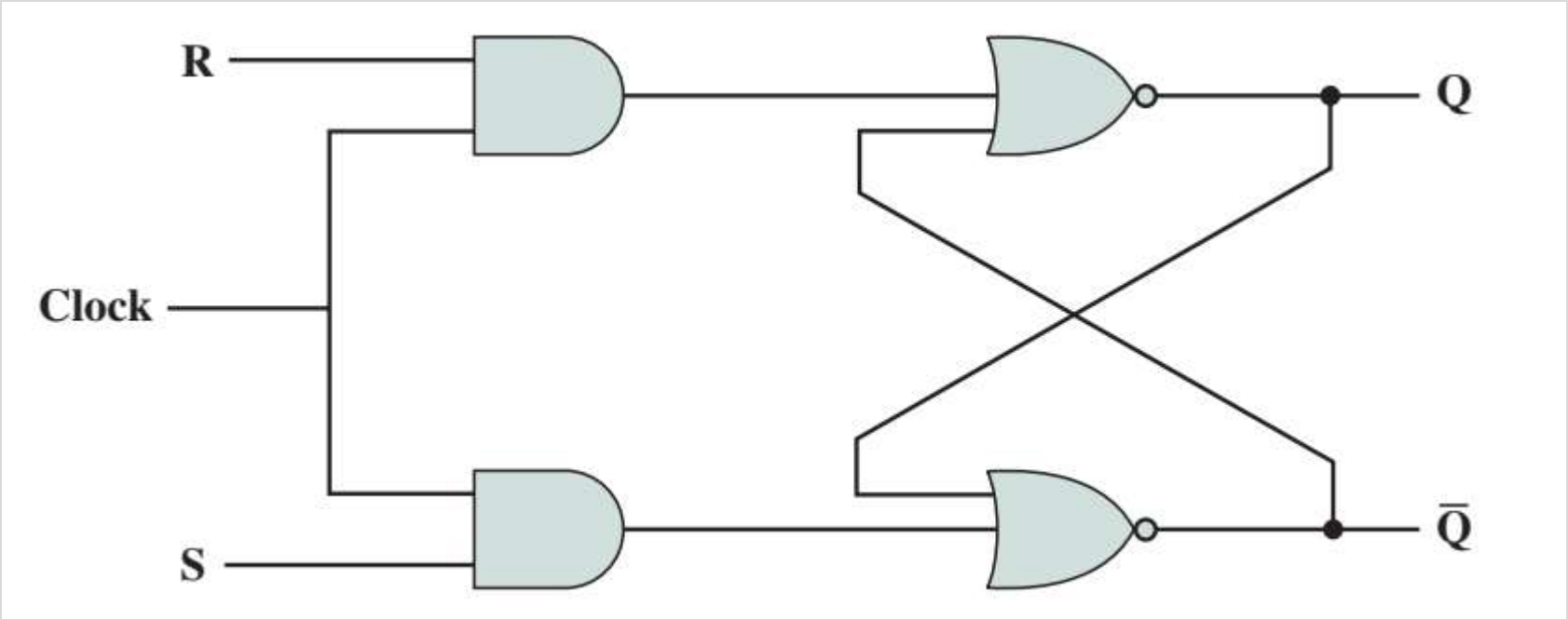
(c) Response to Series of Inputs										
t	0	1	2	3	4	5	6	7	8	9
S	1 →	0	0	0	0	0	0	0 →	1 →	0
R	0	0	0 →	1	0	0 →	1 →	0	0	0
Q_{n+1}	1	1	1	0	0	0	0	0	1	1

Clocked S-R Flip-Flop

- S-R latch
  - Output changes in **response** to a change in input
  - Brief time **delay**
  - **Asynchronous**

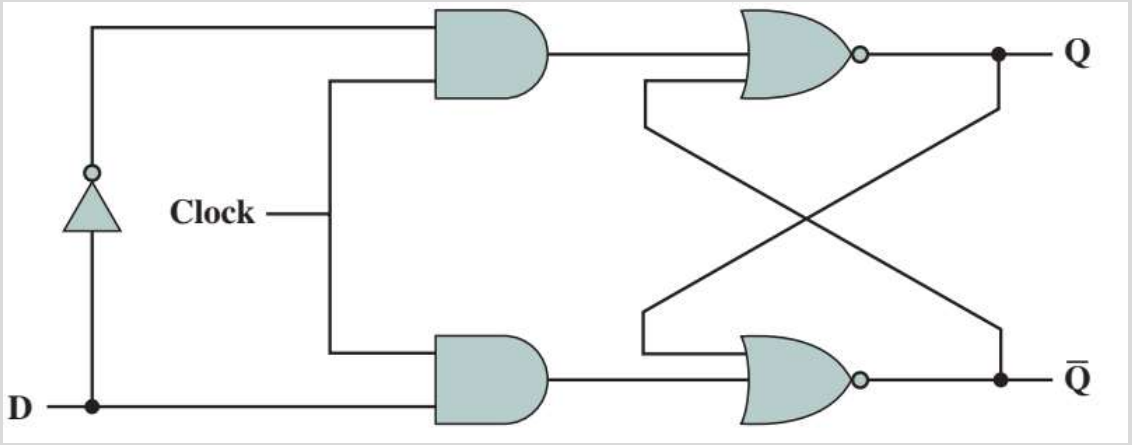
Clocked S-R Flip-Flop

- Events in digital computer are **synchronised** according to clock pulse
  - Clocked S-R flip-flop



D Flip-Flop

- A single input ensures that **S=R=1 never occurs**
  - Inverter ensures (non-clock) inputs to the AND gates are opposite
- Also called **data** flip-flop



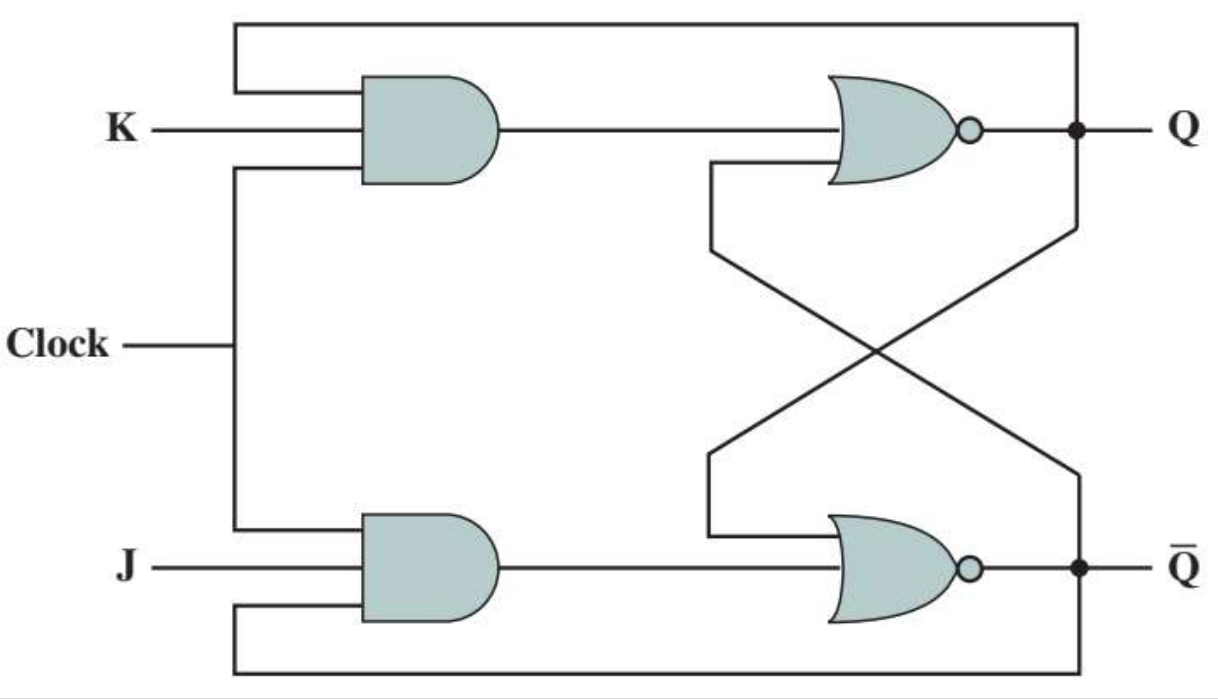
D	Q_{n+1}	
0	0	Reset
1	1	Set

J-K Flip-Flop

- Similar to S-R flip-flop
- **All combinations** of input values are **valid**

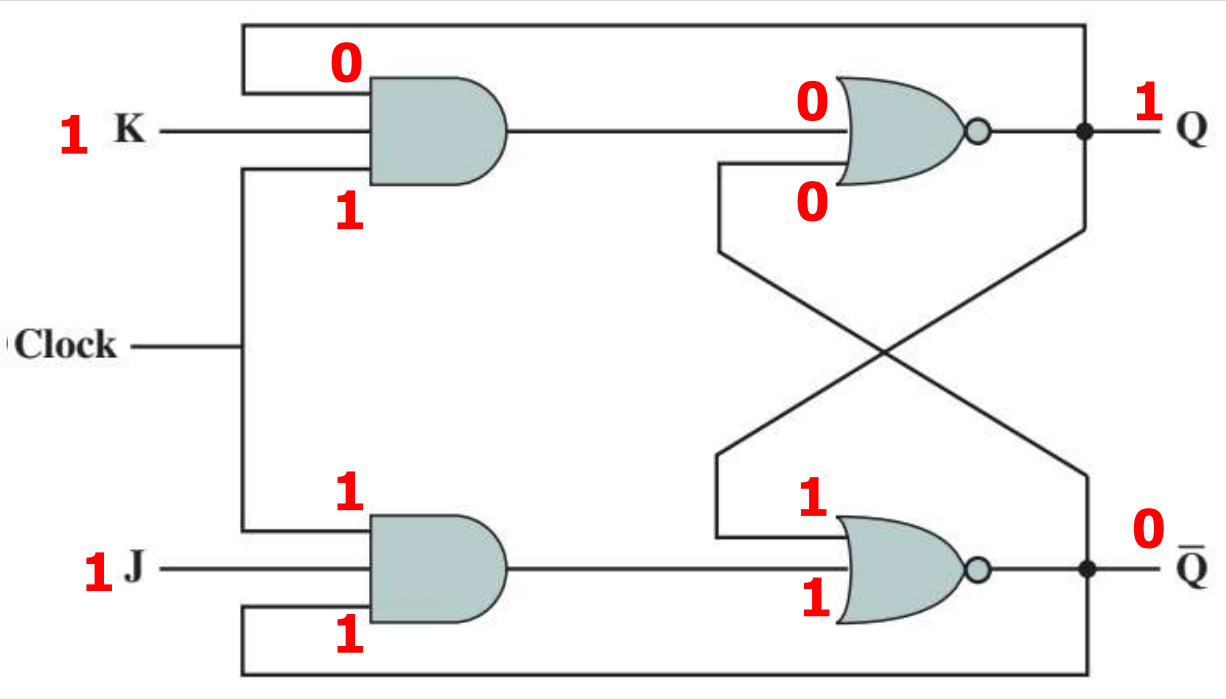
J-K Flip-Flop

- When J=K=1 it acts as a **toggle** function, the output is a **reverse** of the present state

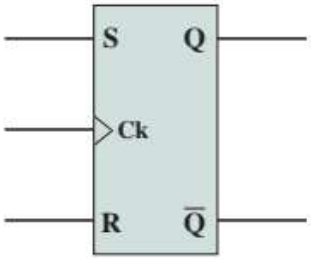
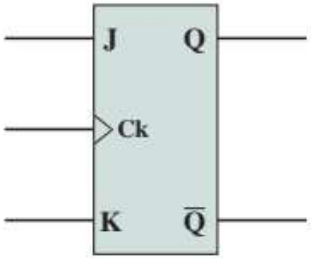
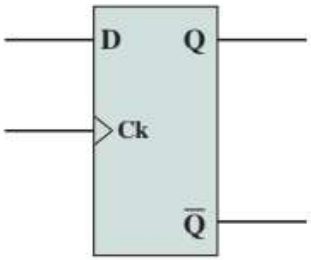


J-K Flip-Flop

- When J=K=1 it acts as a **toggle** function, the output is a **reverse** of the present state



S-R vs. J-K vs. D Flip-Flips

Name	Graphical Symbol	Truth Table
S–R		S R Q_{n+1} 0 0 Q_n 0 1 0 1 0 1 1 1 –
J–K		J K Q_{n+1} 0 0 Q_n 0 1 0 1 0 1 1 1 $\overline{Q_n}$
D		D Q_{n+1} 0 0 1 1

S = to set
R = to reset
S = R =1, not ALLOWED

J = to set
K = to reset
S = R =1, it will toggle the Q value

Input value will be channelled to output

Outline

- Sequential Circuits
  - Flip-Flops
  - Registers
  - Counters
- Programmable Logic Devices
  - Programmable Logic Array
  - Field-Programmable Gate Array

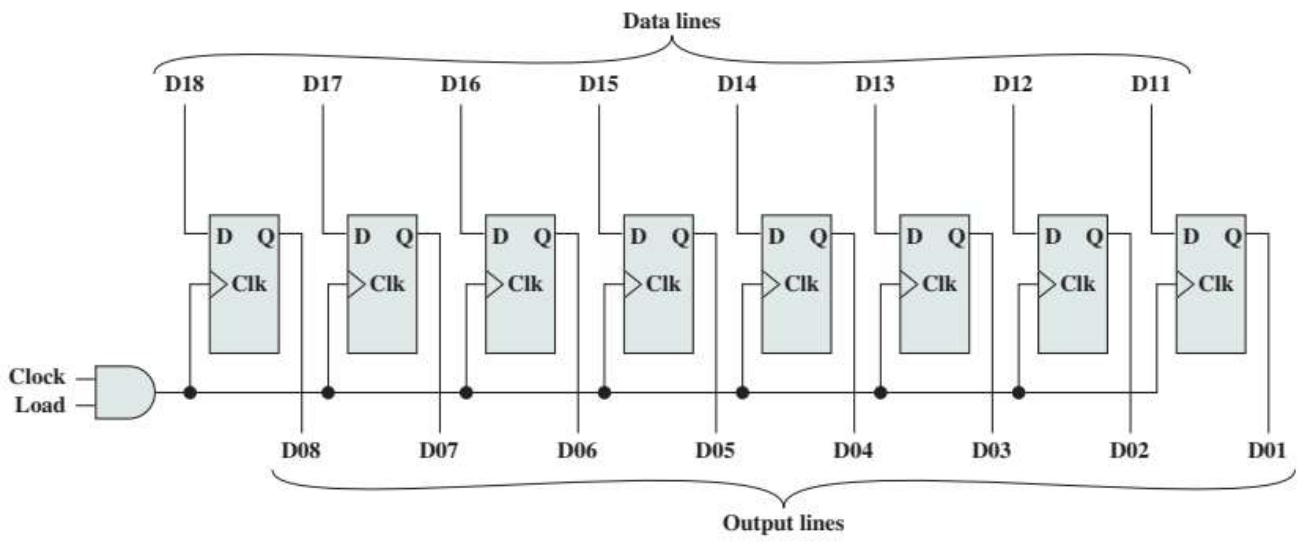
Registers

- Digital circuit within CPU to **store** one or more bits of data

- Two types:
  - **Parallel** registers
  - **Shift** registers

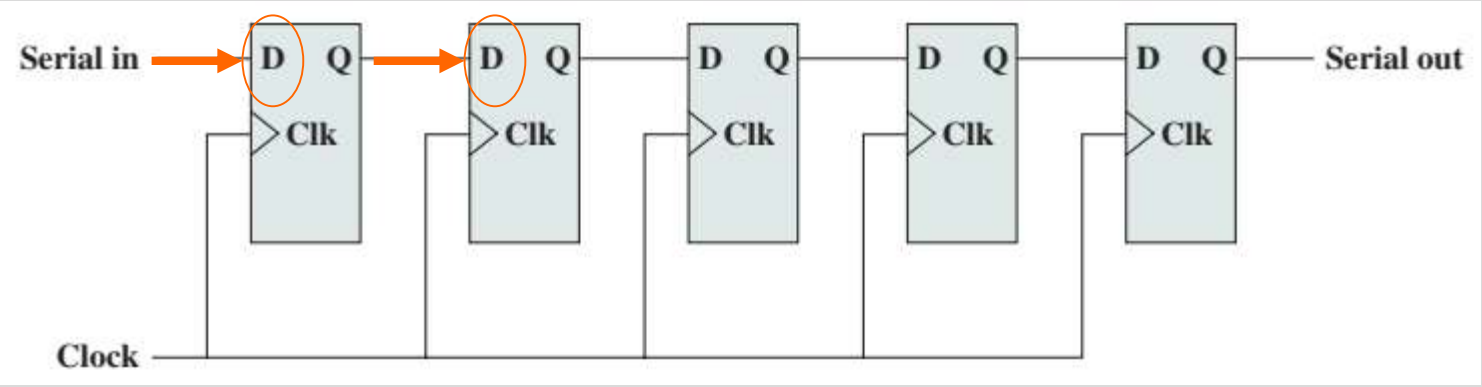
Parallel Registers

- Supports **simultaneous** reading or writing to a set of 1-bit memories
- Uses **D** flip-flops



Shift Registers

- Accepts/**transfers** information **serially**
  - Use (clocked) D flip-flops (or even S-R)
  - Input only to left-most flip-flop
  - Data are shifted one position at every clock pulse
  - Use to interface to serial I/O devices, perform logical shift/rotate functions (ALU), etc.



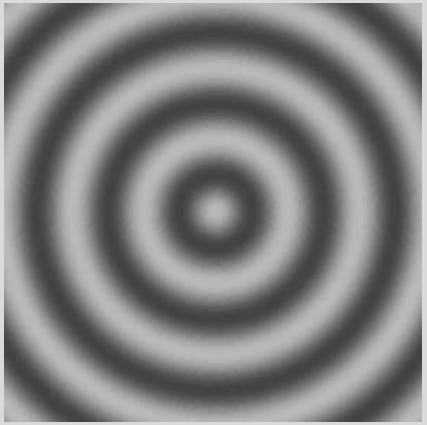
Counters

- Register which increment its value by modulo of its capacity
- *n* flip-flop has the capacity up to $2^n - 1$
  - Beyond maximum, is set to 0
- E.g. CPU's program counter

4 flip flops = $2^4 - 1 = 15$ counters

Ripple Counter

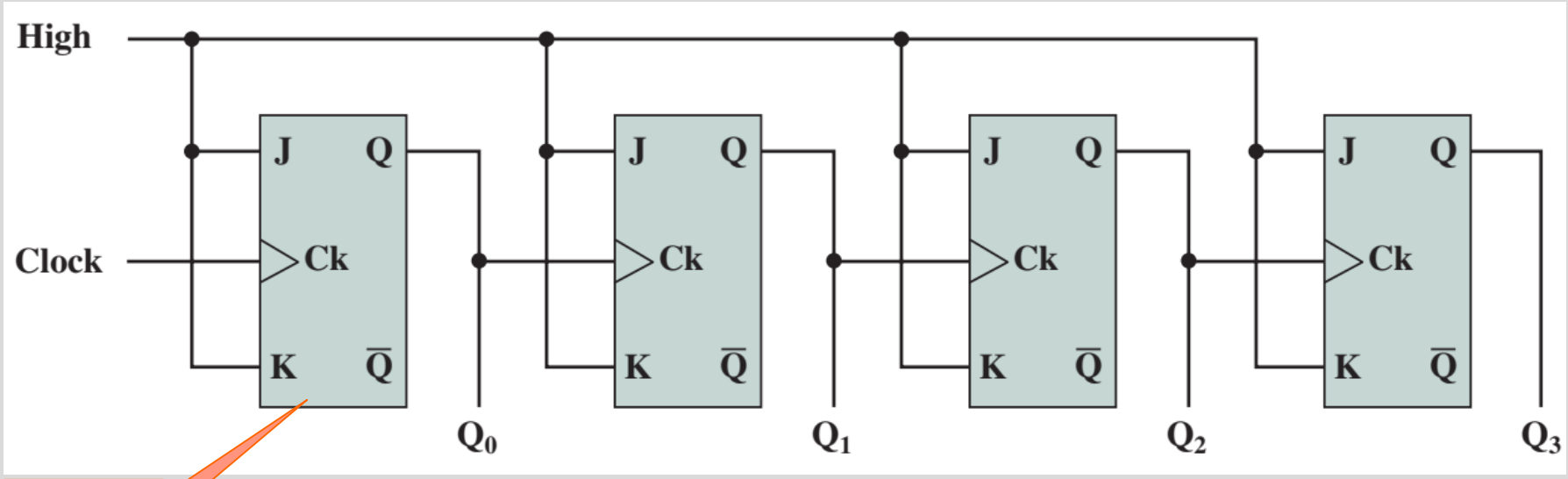
- **Slow**
  - Output of one flip-flop **triggers a change in the next** flip-flop (ripple effect)
- The counter is incremented at each clock pulse (0000, 0001,0010,0011,…1111, 0000)



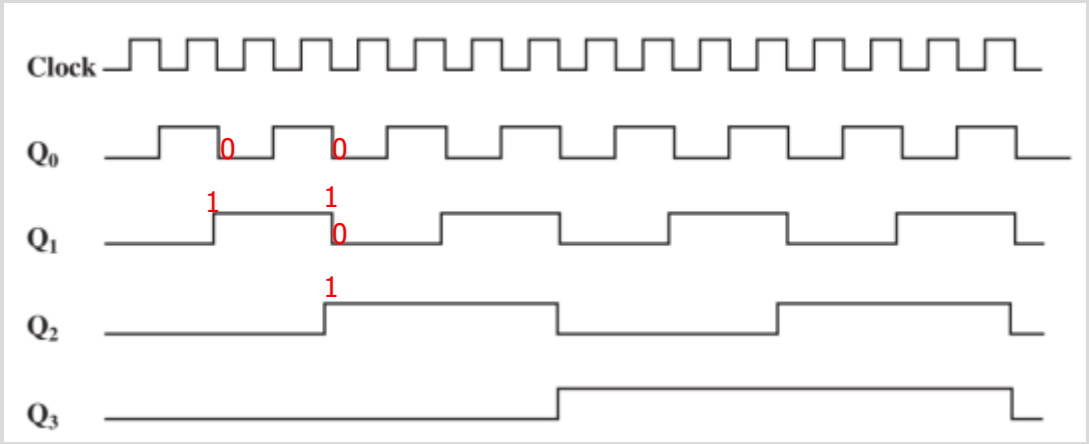
Ripple Counter

- **J and K inputs** to each flip-flop are **held at 1**
  - When there is clock pulse, output Q is **inverted** (0 to 1, 1 to 0)
- Q_0 is least significant bit

Ripple Counter

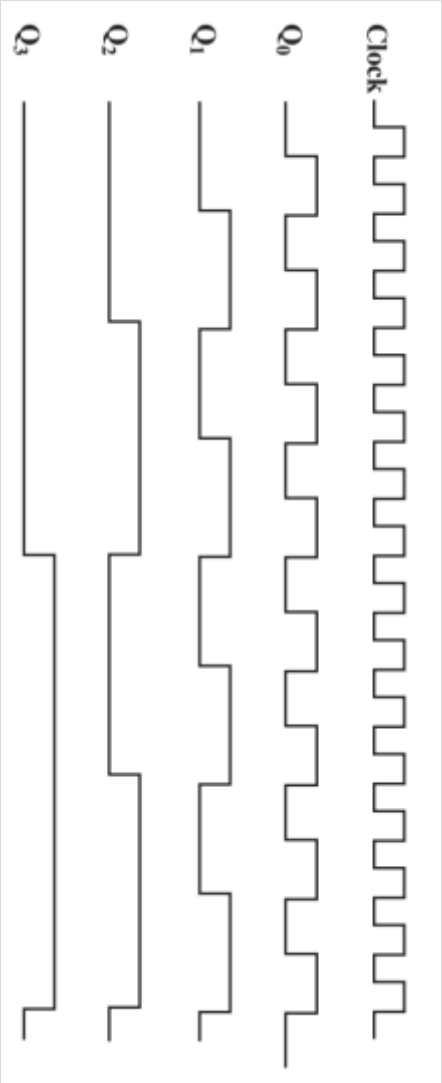


Will be toggling most often



When Q0 becomes 0, then only Q1 becomes 1

Ripple Counter



Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Q0 toggle the most often

Synchronous Counter

- **Faster**
  - All flip-flops **change state at the same time**
  - CPUs use synchronous counters
- E.g. 3-bit counter using J-K flip-flop
  - 3 flip-flops (A, B, and C – A is least significant bit)
  - Construct truth table

Synchronous Counter

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q}_{n+1}$

Q_n	J	K	Q_{n+1}
0	0	d	0
0	1	d	1
1	d	1	0
1	d	0	1

Synchronous Counter

MSB LSB

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka	
							S	R	
0	0	0	0	d	0	d	1	d	Set & Toggle
0	0	1	0	d	1	d	d	1	Toggle & Reset
0	1	0	0	d	d	0	1	d	
0	1	1	1	d	d	1	d	1	
1	0	0	d	0	0	d	1	d	
1	0	1	d	0	1	d	d	1	
1	1	0	d	0	d	0	1	d	
1	1	1	d	1	d	1	d	1	

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_{n+1}}$

Q_n	J	K	Q_{n+1}
0	0	d	0
0	1	d	1
1	d	1	0
1	d	0	1

Synchronous Counter

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

$Jc = BA$

C \ BA	00	01	11	10
0			1	
1	d	d	d	d

$Kc = BA$

C \ BA	00	01	11	10
0	d	d	d	d
1			1	

$Jb = A$

C \ BA	00	01	11	10
0		1	d	d
1		1	d	d

$Kb = A$

C \ BA	00	01	11	10
0	d	d	1	
1	d	d	1	

$Ja = 1$

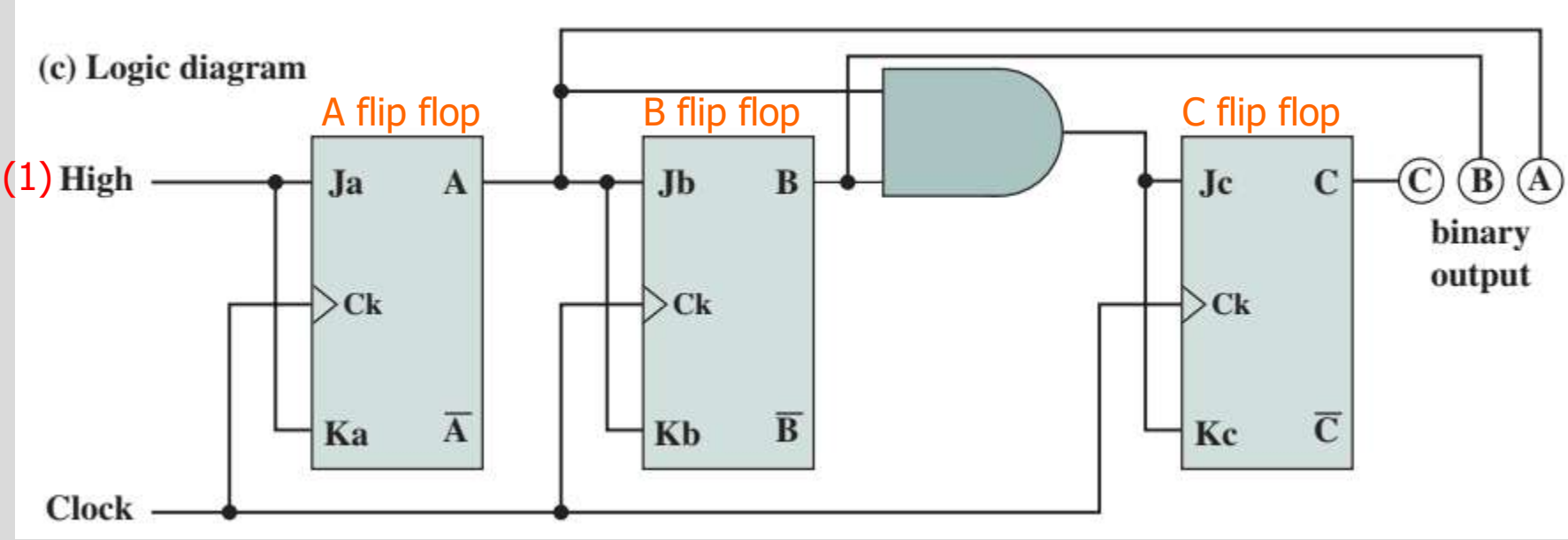
C \ BA	00	01	11	10
0	1	d	d	1
1	1	d	d	1

$Ka = 1$

C \ BA	00	01	11	10
0	d	1	1	d
1	d	1	1	d

Synchronous Counter

- Ja = 1
- Ka = 1
- Jb = A
- Kb = A
- Jc = BA
- Kc = BA



Outline

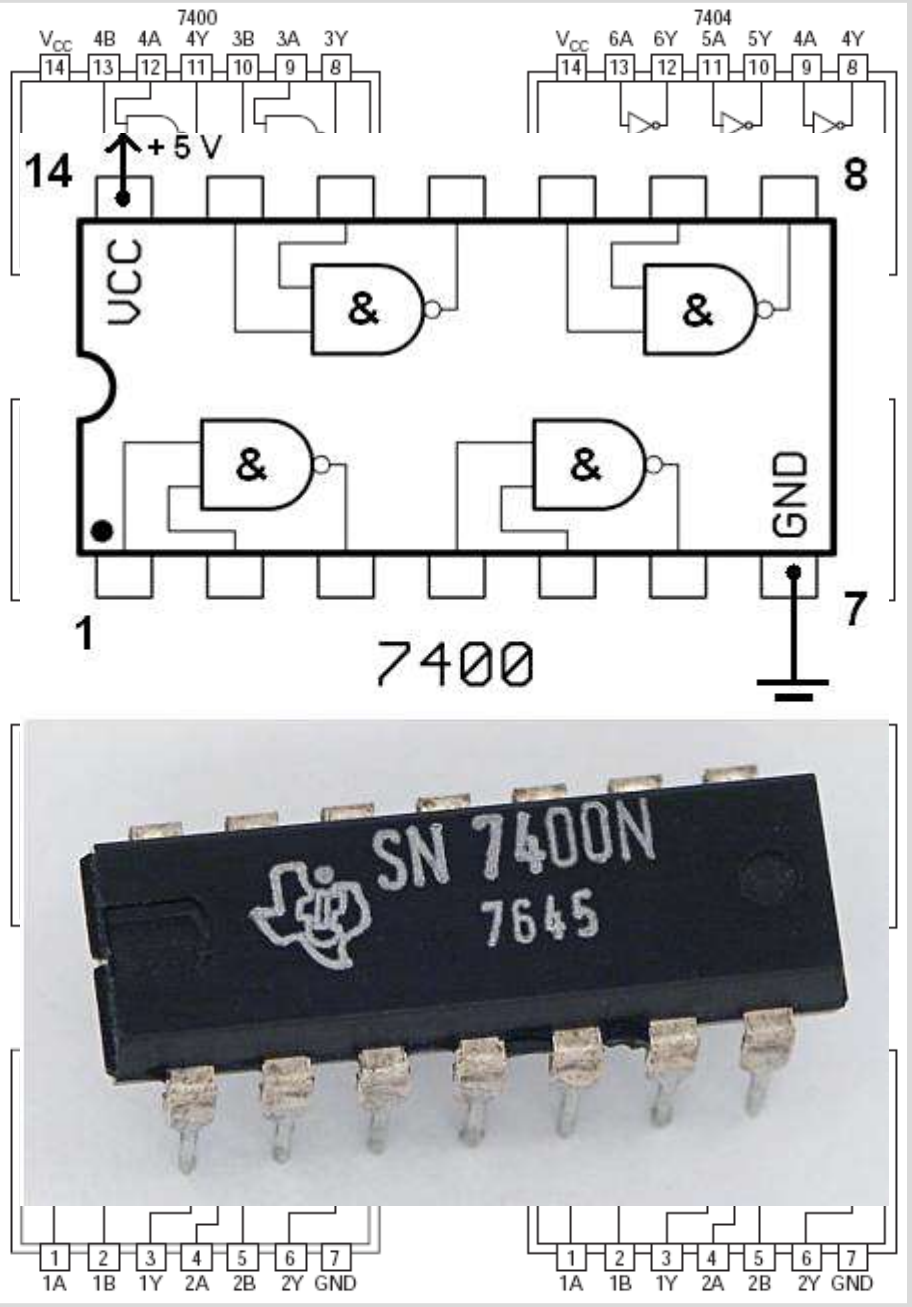
- Sequential Circuits
  - Flip-Flops
  - Registers
  - Counters
- Programmable Logic Devices
  - Programmable Logic Array
  - Field-Programmable Gate Array

Programmable Logic Devices

- So far, individual **gates are building blocks**
  - Any function can be realised

Programmable Logic Devices

- Early integrated circuits using SSI provided 1 to 10 gates on a chip
  - To construct a logic function, **a number of these chips are laid out** and the appropriate pin interconnections are made



Programmable Logic Devices

- Increasing levels of integration made it possible to put **more gates on a chip** and to make gate interconnections on the chip as well.
- Advantages:
  - Decreased cost and size
  - Increased speed

Programmable Logic Devices

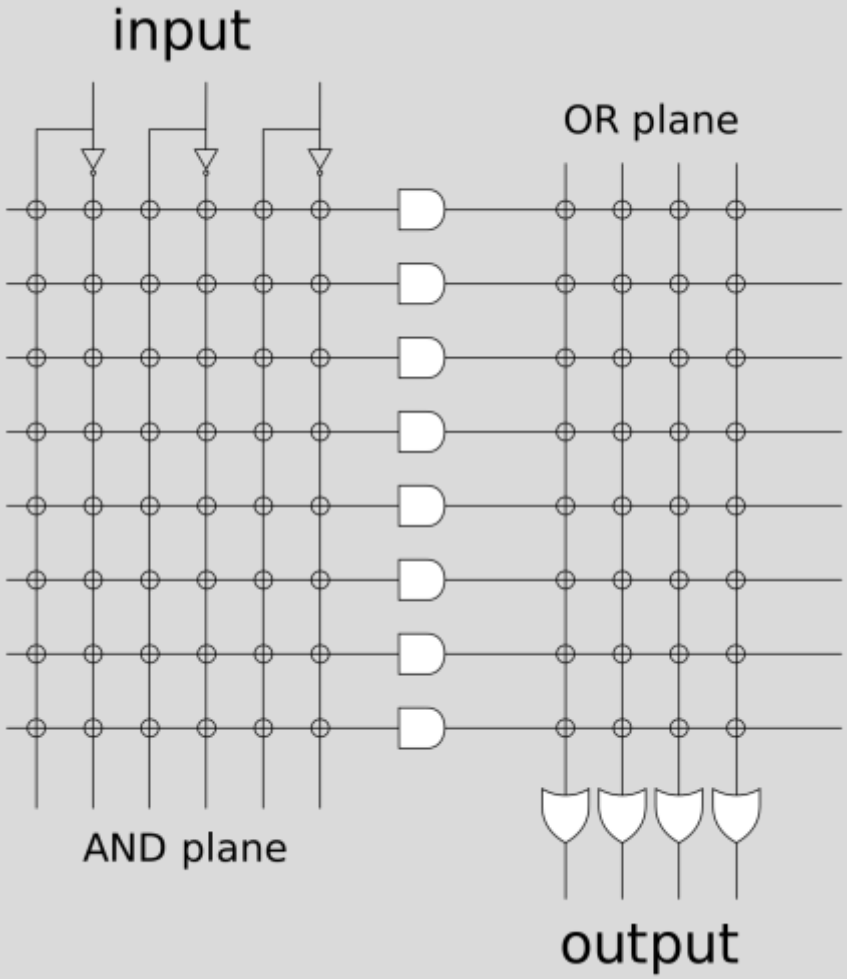
- However,
  - For each logic function, layout of gates and **interconnections must be designed**
  - **Cost and time** involved for **custom chips** design is high

Programmable Logic Devices

- Thus,
- it becomes attractive to develop a **general-purpose chip** that can be readily adapted to specific purposes
- This is the intent of the programmable logic device (PLD)

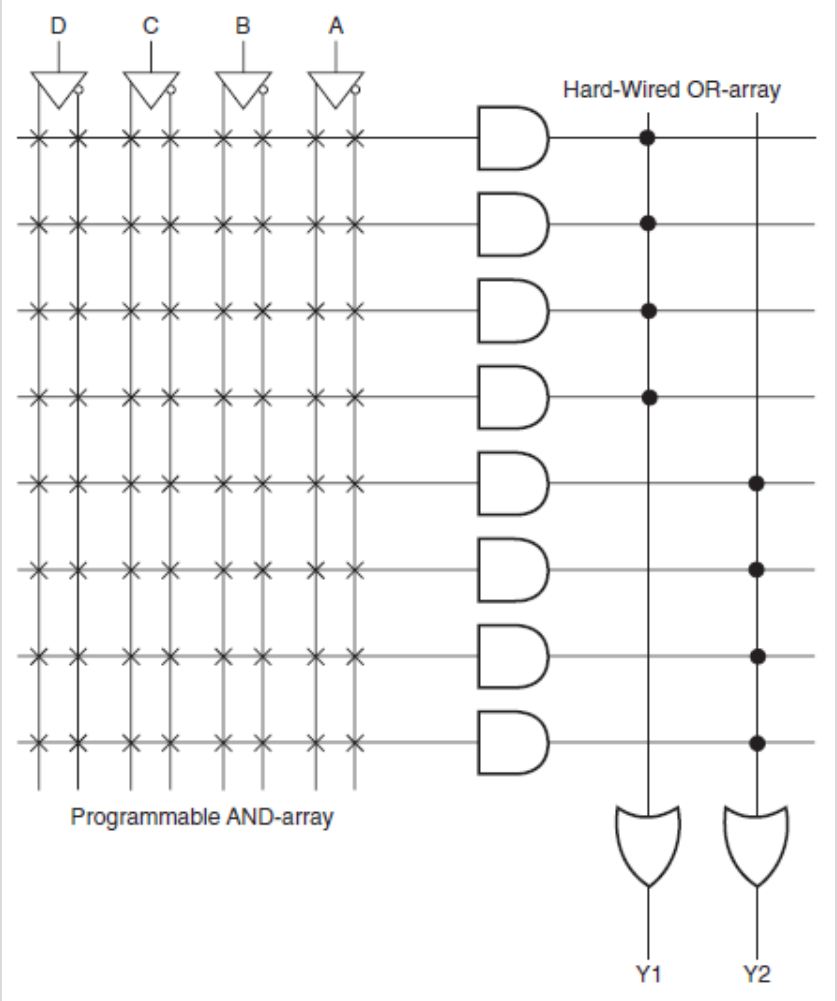
Programmable Logic Devices

- Programmable Logic Array (PLA)
  - A relatively small PLD that contains **two levels** of logic, an **AND-plane** and an **OR-plane**, where **both levels are programmable**



Programmable Logic Devices

- Programmable Array Logic (PAL)
  - A relatively small PLD that has a **programmable AND-plane** followed by a **fixed OR-plane**



Programmable Logic Devices

- **Simple** PLD (SPLD)
  - A PLA or PAL
- **Complex** PLD (CPLD)
  - A more complex PLD that consists of an arrangement of multiple SPLD-like blocks on a single chip

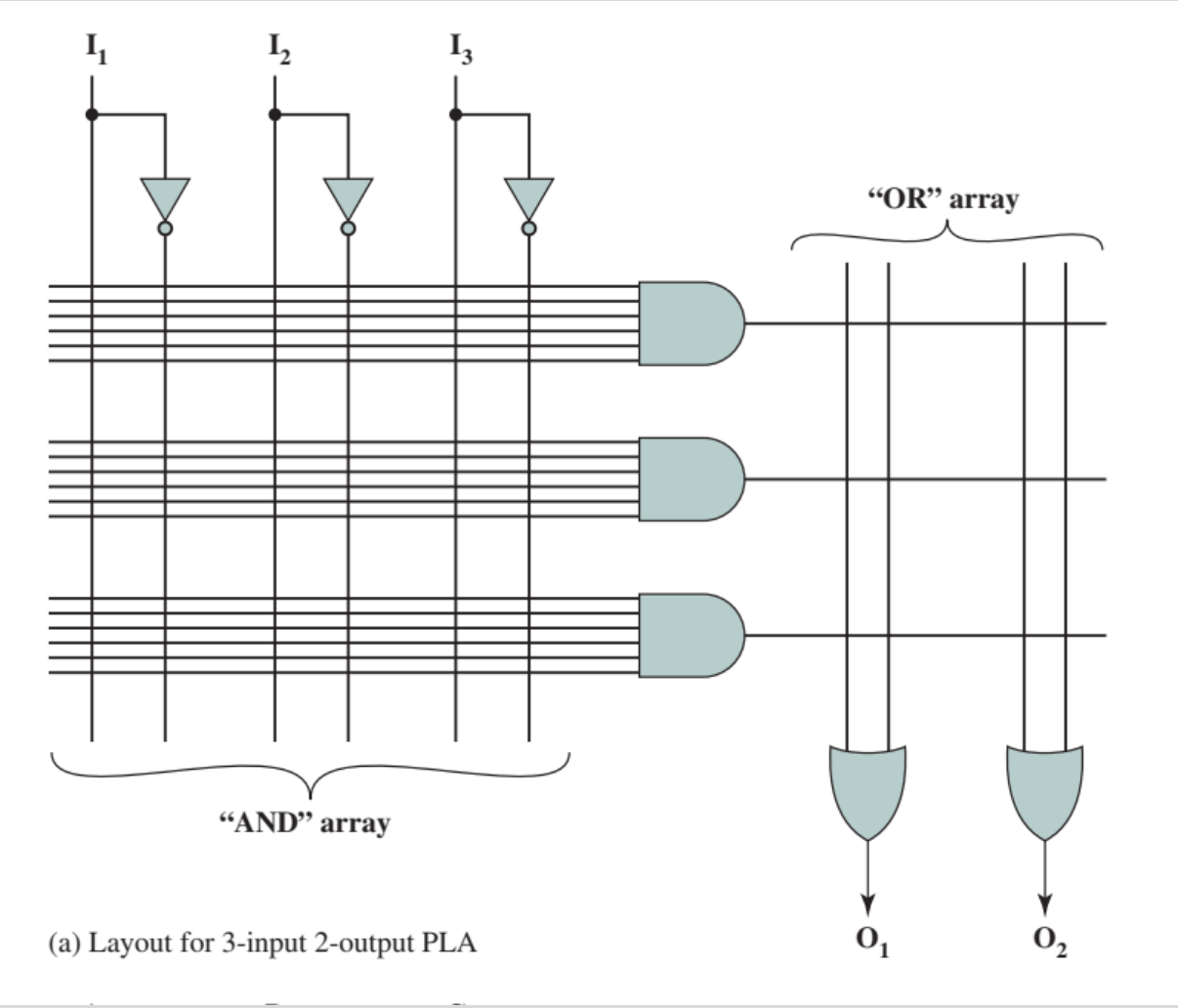
Programmable Logic Devices

- **Field-Programmable Gate Array (FPGA)**
  - A general structure that allows very high logic capacity
  - Whereas CPLDs feature logic resources with a wide number of inputs (AND planes), FPGAs offer more narrow logic resources
  - FPGAs also offer a higher ratio of flip-flops to logic resources than do CPLDs
- **Logic Block**
  - A relatively small circuit block that is replicated in an array in an FPD

Programmable Logic Array

- Based on fact that any Boolean function can be **expressed in SOP**
- Consists of NOT, AND, and OR gates
- Each chip input is NOTed
- Output of each AND gate is available to each OR gate
- Output of each OR gate is a chip output
- Appropriate connects are made to implement a particular SOP expression

Programmable Logic Array

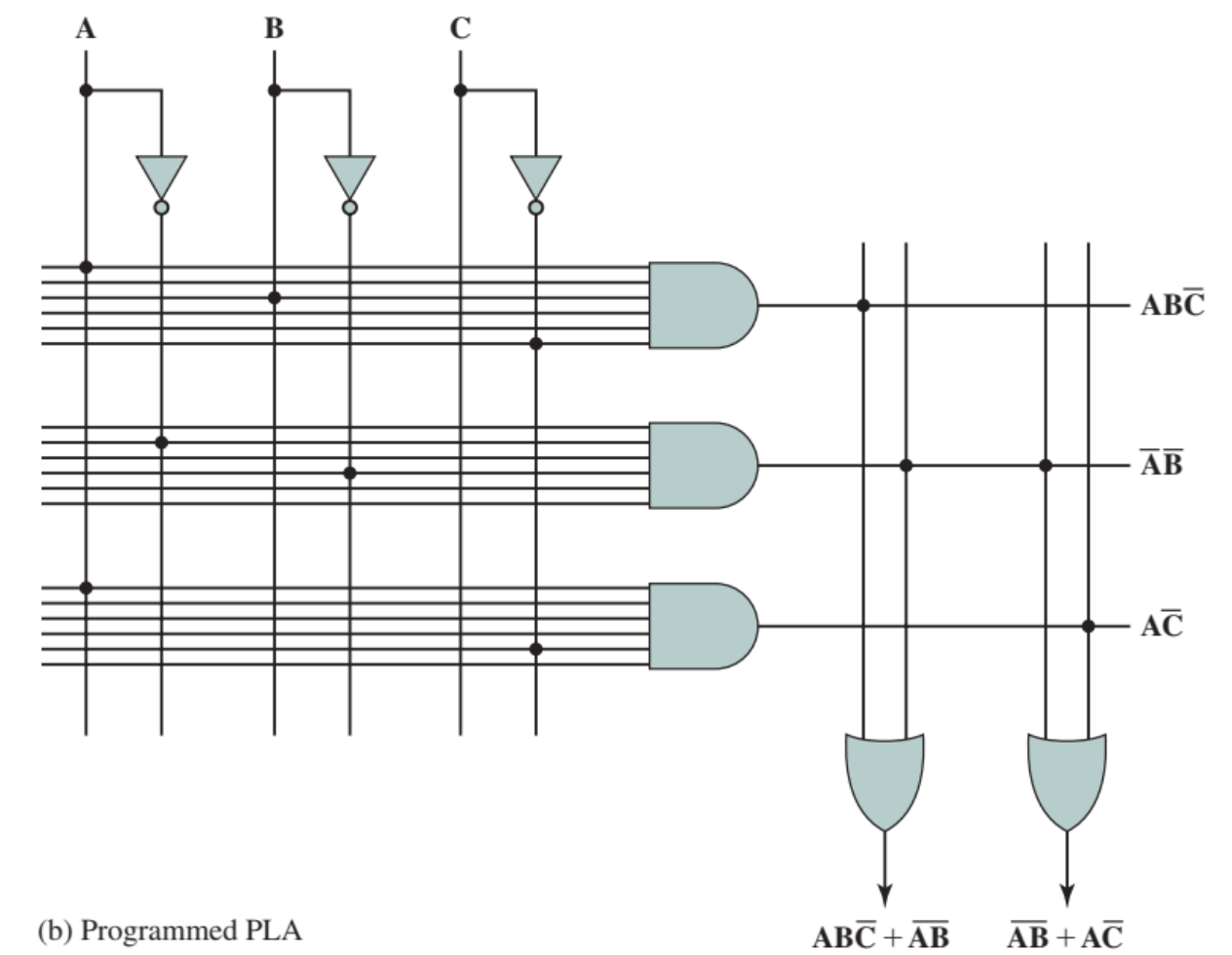


(a) Layout for 3-input 2-output PLA

Programmable Logic Array

- PLAs manufactured in 2 ways
  - Every possible connection is made through a **fuse** at every intersection
    - Undesired connections are removed by blowing the fuses
    - Field Programmable Logic Array
  - Proper connections made during fabrication
- PLAs provide flexible, inexpensive way of implementing digital logic functions

Programmable Logic Array



(b) Programmed PLA

Field-Programmable Gate Array

- PLA is an example of SPLD
- Problem is that the structure of programmable logic planes **grows too quickly in size** as number of inputs increased

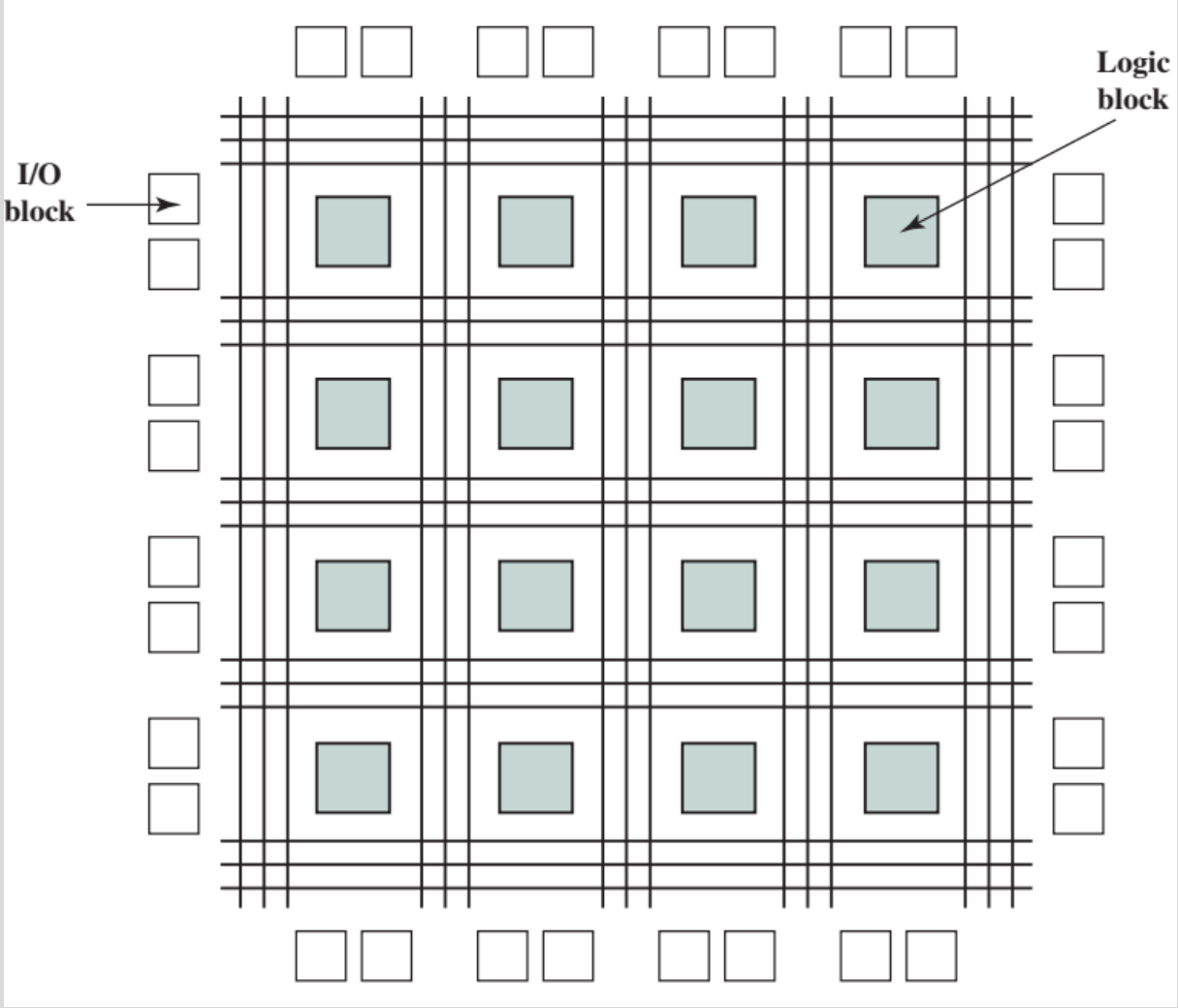
Field-Programmable Gate Array

- Solution is to integrate **multiple SPLD onto single chip**
  - Provide interconnect to programmably connect the SPLD blocks together
  - Complex PLDs (CPLDs)
  - Most important type is FPGA

Field-Programmable Gate Array

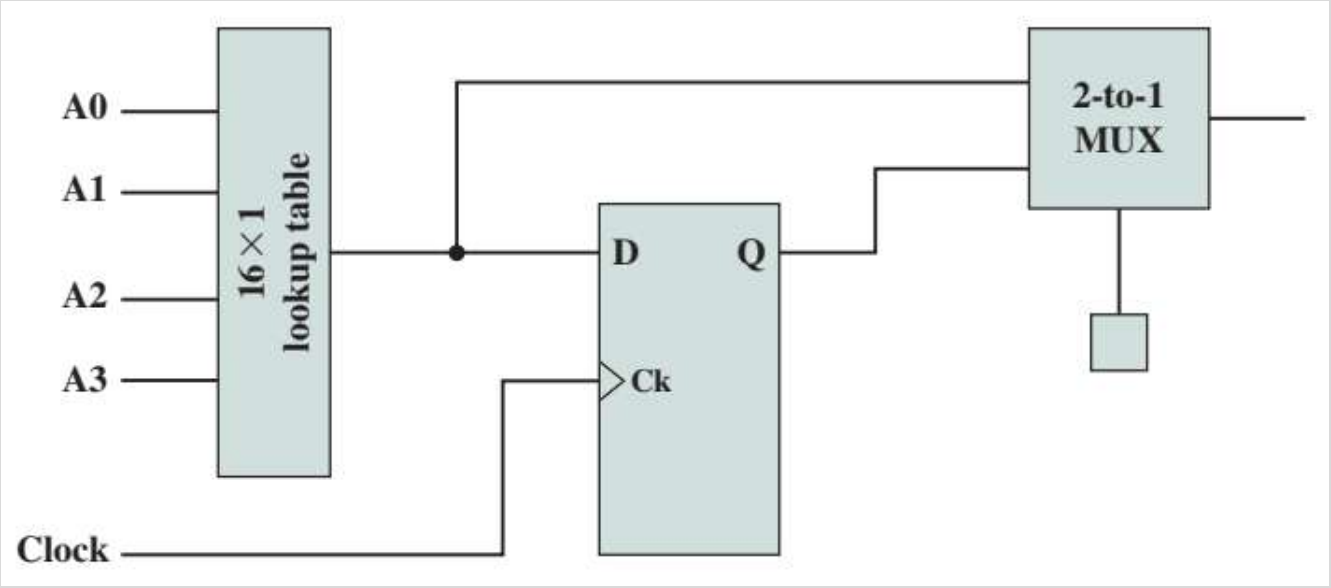
- FPGA consists of an **array of uncommitted circuit elements**
- Key components
  - Logic block – computation takes place
  - I/O block – connects I/O pins to circuitry on chip
  - Interconnect – establish connections among I/O blocks and logic blocks

Field-Programmable Gate Array



Field-Programmable Gate Array

- Logic block is either combinational or sequential circuit
  - Programming is done by downloading contents of truth table



Field-Programmable Gate Array

http://en.wikipedia.org/wiki/Field-programmable_gate_array

Image source: https://www.flickr.com/photos/29143375@N05/4582236533